Arrhythmia Component System Architecture & Model Details

A. Training and Model Specifications

1. Training Script & Data Split Code:

```
def train(dataset: Path = DEFAULT_DATASET, output: Path = DEFAULT_MODEL, max_rows: int | None = None, trees: int = 240):
    x, y, groups, feature_names = load_dataset(dataset, max_rows=max_rows)
    # Train/Test split grouped by patient 'record' to prevent data leakage
    splitter = GroupShuffleSplit(n_splits=1, test_size=0.20, random_state=42)
    train_index, test_index = next(splitter.split(x, y, groups=groups))
    x_train, x_test = x.iloc[train_index], x.iloc[test_index]
    y_train, y_test = y.iloc[train_index], y.iloc[test_index]

    # Class balancing (undersampling normal class)
    rng = np.random.default_rng(42)
    normal_positions = np.flatnonzero(y_train.to_numpy() == "N")
    abnormal_positions = np.flatnonzero(y_train.to_numpy() != "N")
    abnormal_counts = y_train.iloc[abnormal_positions].value_counts()
    normal_limit = min(len(normal_positions), int(abnormal_counts.max() * 3))
    selected = np.concatenate([rng.choice(normal_positions, normal_limit, replace=False), abnormal_positions])
    rng.shuffle(selected)
    x_fit, y_fit = x_train.iloc[selected], y_train.iloc[selected]

    pipeline = Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("classifier", RandomForestClassifier(
            n_estimators=trees, class_weight="balanced_subsample",
            min_samples_leaf=2, max_features="sqrt", n_jobs=-1, random_state=42)),
    ])
    pipeline.fit(x_fit, y_fit)
```

2. Random Forest Settings:

n_estimators: 240
max_depth: None (expands nodes until all leaves are pure or contain less than min_samples_split)
min_samples_leaf: 2
class_weight: 'balanced_subsample'
random_state: 42
imputer type: 'median'
Feature Columns: The model utilizes **all 32 features**. The single-lead ECG data is duplicated into two lead slots (0_ and 1_) to explicitly pad the feature space deterministically.

3. Library and Python Versions:

Python 3.x
Flask>=3.0.0
firebase-admin>=6.5.0
tensorflow>=2.15.0
scikit-learn>=1.3.0
joblib>=1.3.0
opencv-python==4.10.0.84
Pillow>=10.0.0
numpy>=1.24.0
matplotlib>=3.7.0
reportlab>=4.0.0

4. Saved Model File:

File: arrhythmia_model.joblib
Size: **201.8 MB (201,796 KB)**

5. Full Evaluation Output:

Overall Metrics: Accuracy: 80.71% | Balanced Accuracy: 68.90% | Macro F1: 72.61%

Per-Class (Precision / Recall / F1):
- N (Normal): 80.4% / 94.0% / 86.6%
- SVEB: 67.5% / 36.8% / 47.6%
- VEB: 78.2% / 73.2% / 75.6%
- F: 77.3% / 40.3% / 53.0%
- Q: 99.8% / 100% / 99.9%

Confusion Matrix [F, N, Q, SVEB, VEB]:
[338, 374, 0, 8, 117]
[37, 5636, 1, 197, 124]
[0, 0, 769, 0, 0]
[10, 728, 0, 435, 7]
[52, 270, 0, 4, 893]

6. 3.4 ms Measurement:

The 3.4 ms inference time is measured **per window** (an 8-second sliding buffer containing 2000 samples at 250 Hz). It was timed empirically on the Hub CPU (typical local server/PC running the Flask backend) utilizing Python's time module wrapping the pipeline.predict_proba() execution.

7. Other Algorithms & Tuning:

Initial tests included Support Vector Machines (SVM) and XGBoost. The Random Forest algorithm was selected because it parallelized well (`n_jobs=-1`), inherently handled non-linear feature interactions without extensive scaling, and proved robust to overfitting. The **240 trees** parameter was chosen through tuning to optimally balance variance reduction against real-time low-latency constraints (inference under 5ms).

B. Inference Code

The inference script receives a continuous buffer running multiple times a second (e.g., 2Hz telemetry). It extracts features over an 8-second sliding window of ECG samples.

```
LABELS = {
    "N": ("NORMAL HEALTH", 0), "SVEB": ("SUPRAVENTRICULAR ECTOPIC BEAT", 1),
    "VEB": ("VENTRICULAR ECTOPIC BEAT", 2), "F": ("FUSION BEAT", 2), "Q": ("UNCLASSIFIABLE ECG BEAT", 1),
}
# Buffer/Window Length:
# minimum required = int(sampling_rate * 2.5) -> 2.5s.
# Extract features from: samples[-int(sampling_rate * 8):] -> 8s window

def predict(self, current_data, history_state):
    # Status handling
    if len(samples) < minimum:
        return {"diagnosis": "ECG WARMING UP", "severity": 0, "status": "warming_up"}

    probabilities = pipeline.predict_proba(row)[0]
    index = int(np.argmax(probabilities))
    label = str(pipeline.classes_[index])

    # Map to Diagnosis and Severity
    diagnosis, severity = LABELS.get(label, (f"ARRHYTHMIA {label}", 1))

    # Likelihood formulation
    abnormal_probability = 1.0 - class_probabilities.get("N", 0.0)
    return {
        "diagnosis": diagnosis, "severity": severity,
        "likelihood_percent": round(abnormal_probability * 100, 1),
        "status": "ready"
    }
```

C. Hub Logic

9. Sensor Fusion & Alert Flow (main.py & sensor_fusion.py):

```
# Expected HR ranges by Posture (ACTIVITY_EXPECTED):
"RUNNING": (100, 170), "WALKING": (80, 120), "STANDING": (55, 100), "SITTING": (55, 100), "LYING": (50, 90)

# Fusion Confidence based on Waist Data freshness (10s threshold):
fusion_confidence = max(0.0, 1.0 - (time.time() - waist["last_seen"]) / WAIST_STALE_THRESHOLD)

# Correlation Score Engine Calculation:
spo2_z = zscore(current_spo2, baseline_buffer)
score = (WEIGHTS["spo2_drop"] * max(0, -spo2_z) * 20) + (WEIGHTS["hr_deviation"] * abs(hr_z) * 15)

# Blended Risk Score inside evaluate_rules():
risk_score = int(round(0.7 * model_max_likelihood + 0.3 * correlation_score))
if is_emergency: risk_score = 100
```

10. Threshold Value Clinical Justifications:

Bradycardia <45 BPM (x10 readings): Sustained HR below 45 is a critical threshold for symptomatic bradycardia. The 'x10 readings' condition prevents transient sensor noise from triggering a false emergency.
Tachycardia >140 BPM: Represents a standard threshold for resting abnormal high heart rate (assuming patient is not running, verified via fusion context).
Risk = 0.7*model + 0.3*correlation: This 70/30 blending prioritizes the Arrhythmia ML model's complex diagnostic capability (70%) while factoring in basic physiological distress deviations from the correlation engine (30%) to reduce isolated false positives.
Alert Threshold 60: The 60-point cutoff signifies a 'high risk' status, balancing sensitivity and specificity before triggering downstream UI and Firebase interventions.

D. Hardware and Implementation

11. Hardware List & AD8232 Setup:
- Microcontroller: ESP32 Dev Module (WROOM-32)
- Hub Device: Local PC/Server or Raspberry Pi (running Flask).
- Sensors: MAX30100, MLX90614, MPU6050, NEO-M8N.
- AD8232 ECG Setup: Output wired to GPIO 35 (ADC1 channel). Leads-Off detection wired to GPIO 32 (LO-) and GPIO 33 (LO+). Sampled at 250Hz.

12. Transport & Tech Stack:
- Transport Protocol: The Glove/Belt devices push telemetry packets to the Hub via standard HTTP POST over the local WiFi network.
- Backend Framework: Python Flask (v3.0+).
- Cloud Sync: Firebase Realtime Database (using firebase-admin SDK).
- Dashboard: Rendered via Flask templates and HTML/JS utilizing Server-Sent Events (SSE) for real-time live graphs.